Activity 4 Progress Report

Database Users Roles Privileges and Access Control

COMP 018 Database Administration
Group 09 - Batumbakal Gym
6 November 2026

Members: Louis Andrei de Guzman; Mark Timothy Casilan. Individual task roles were not supplied.

Repository: https://github.com/louisandreidg/Batumbakal-Gym
Class tracker: https://docs.google.com/spreadsheets/d/1BClCa9rEs9o-D9Rp3tUZVhECipJ234zX/edit?gid=2123136180

Submission status

This package documents the four projects confirmed for the current testing work: School Event Management, Applicant Management, SHIORI Cafe and ReserveIT. The working PostgreSQL projects were tested with separate runtime, reporting and owner accounts. SHIORI remains blocked because its active application still uses MySQLi.

The SQL files, access-control plans, actual output and feedback drafts are prepared. Feedback has not been sent and class-sheet updates have not been submitted by the assistant. Testing every other class project has not been established. Earlier Dropboxed and Pawpaw records are included separately with their evidence limitations.

Links for submission

Access-control folder: https://github.com/louisandreidg/Batumbakal-Gym/tree/main/db-access-control
PDF link after publication:
https://github.com/louisandreidg/Batumbakal-Gym/blob/main/db-access-control/Activity4-Progress-Report.pdf

Evidence policy

Passed operations are supported by actual command output or application responses. Configuration audits show grants and objects only. Reconstructed SQL is labeled and is not presented as an original execution log. The report contains no real database passwords; local encrypted credential files are excluded.

Projects and setup results

Project and snapshot	Database setup result	Application and evidence scope
G02 School Event Management main ZIP; commit unavailable	Success with issues	Database and Flyway migrations worked. Application CRUD, login, role restrictions and database permissions passed. Stale role documentation and the missing documented constraint-test script remain.
G07 Applicant Management System main / 2ccec581b94708d5a8f864c353281454dc725fcd	Success	Database setup and application startup worked. Basic CRUD, validation, CSRF and restricted database operations passed.
G10 SHIORI Cafe main / c8ee5a2daaac8d84d06e9abd70149763cb62e8a6	Failed	The application still uses MySQL/MySQLi. PostgreSQL integration is unfinished, blocking PostgreSQL application and account testing.
G11 ReserveIT main / dd466cabb7c9628fac386a1e55840fd5f0303334	Success	Database setup and application startup worked. Requester, staff and administrator workflows worked; column restrictions, read-only access and prohibited database operations were verified.

Project repositories

G02 School Event Management: https://github.com/JosephEmmanuelBabas/School_Event_Management_Web_Appli

G07 Applicant Management System: https://github.com/mang2mas/grp07-applicant-management-system

G10 SHIORI Cafe: https://github.com/LeeJnsn27/SHIORIcafe-new

G11 ReserveIT: https://github.com/sev-villanueva/ReserveIT-Classroom-and-Material-Reservation-System-

Objects and initialization

Project	Objects and setup
G02	grp02_event_management_db / school_event_management_db. Nine application tables plus Flyway history and identity sequences. Original Flyway V1-V3 applied as event_owner; Hibernate validates schema.
G07	grp07_applicantdb / public. department, position, applicant, application_status, application, interview, requirement, applicant_requirement; serial sequences and constraints. Original schema then sample seed applied as grp07_owner.
G10	PostgreSQL database and schema are planned. Active PHP connection and endpoints still use MySQLi; no PostgreSQL setup success is claimed.
G11	ReserveIT / public. department, app_user, staff, room, material, reservation, reservation_material, penalty, serial sequences and btree_gist. Original manage.py account setup, schema, grants and seed succeeded.

Access control plans

Application roles describe people using a website. PostgreSQL accounts describe the database connections. The application runtime is shared; application code enforces the end-user role matrix. No ordinary runtime, reporting or maintenance account has SUPERUSER, CREATEDB or CREATEROLE.

Project / login	Group role and responsibility	Required privileges
02 / event_owner	Object owner; no privilege group Setup and Flyway migrations	Owns schema and objects; schema DDL. Not SUPERUSER, CREATEDB or CREATEROLE.
02 / event_app	grp02_runtime_grp NOLOGIN Shared runtime for both application roles	SELECT role; SELECT/INSERT/UPDATE user_account; CRUD six operational tables; SELECT/INSERT audit_log; sequence USAGE.

Project / login	Group role and responsibility	Required privileges
02 / grp02_readonly	grp02_readonly_grp NOLOGIN Inspection and reporting	SELECT role, venue, event only. No account-table access, writes, sequence or schema CREATE rights.
07 / grp07_owner	Database/object owner Provision schema and sample data	Owns database and tables. No SUPERUSER, CREATEDB or CREATEROLE.
07 / grp07_app	grp07_runtime_grp NOLOGIN Application runtime	CRUD on operational tables, SELECT only application_status; sequence USAGE.
07 / grp07_readonly	grp07_readonly_grp NOLOGIN Reporting/inspection	SELECT on the eight application tables; no writes or DDL. This is a database role, not a web login.
11 / reserveit_owner	Database/object owner Setup only	Owns database/objects; schema DDL. No SUPERUSER, CREATEDB or CREATEROLE.
11 / reserveit_app	app_runtime_grp NOLOGIN Shared requester/staff/admin runtime	SELECT and required INSERT; only documented UPDATE columns; DELETE only room/material; required sequence USAGE.
11 / reserveit_reporter	app_readonly_grp NOLOGIN Read-only reporting	Approved tables and approved people columns; cannot read emails/password hashes or modify data.
11 / reserveit_dba	app_admin_grp NOLOGIN Data maintenance	Table CRUD and sequence maintenance; no schema DDL, role/database creation or superuser rights.
10 / Not provisioned	Not implemented for testing Planned PostgreSQL roles	Cannot validate a runtime account against PostgreSQL while active code uses MySQLi.

Least privilege and database isolation

Runtime accounts do not own application tables. Owner accounts are used for schema creation and migrations. Reporting accounts receive approved SELECT access; ReserveIT restricts sensitive people-table columns. Group 2 audit_log allows SELECT and INSERT only. Group 7 application_status is read-only to the runtime. ReserveIT limits runtime updates to documented columns and keeps reservation and penalty history from being deleted.

PUBLIC access was restricted on the newly created project databases. Existing unrelated databases had PUBLIC CONNECT grants. To preserve those projects, connection-policy rules were added for only the new tested accounts. Each account can authenticate to its intended project database and is rejected for other databases. This is a pg_hba connection control; a privilege-inspection function can still report inherited PUBLIC CONNECT elsewhere. Actual connection attempts, not that function alone, were used to verify isolation.

The added connection rules were checked for parse errors and reloaded. Previous rules were preserved. Only project-specific policy snippets are in this repository; full local server backups are excluded. Reproduce the snippets on the test server or do not assume the same isolation result.

Sequence and future-object permissions were reviewed. Required inserts exercised generated identifiers. Group 2 and ReserveIT use explicit object grants. Group 7 has default SELECT for its reporting group under grp07_owner; future runtime access still needs review. No automatic blanket write grant for future objects was added.

SQL scripts and provenance

Folder	Scripts and how they were established
group-02	setup.sql is a reusable reconstruction. access-control.sql is the executed group-grant and direct-grant removal script. group02-pg_hba.conf.snippet records the applied isolation addition. Original Flyway files stay in the owning repository.
group-07	setup.sql and grants.sql reconstruct the verified final configuration. Original db/01_schema.sql and db/02_seed.sql from the owning repository are prerequisites. pg_hba.snippet records the isolation addition.
group-10	No fabricated PostgreSQL setup or role script. source-review.txt records the source blocker and reviewed snapshot.
group-11	roles_setup.sql and grants.sql are attributed unmodified source copies. Original manage.py substitutes password and database placeholders. pg_hba.snippet records the additional local isolation policy.

Folder	Scripts and how they were established
group-01 / group-03	Supplementary reconstruction of audited direct grants only. It is not a verified least-privilege implementation or an original execution log.

Verification approach and application behavior

Group 2 backend startup, Flyway and schema validation passed. Its supplied authenticated API suite exercised login, CSRF, role restrictions, CRUD, event overlap, schedules, attendance and reports. Group 7 pages and department CRUD, duplicate/required-field validation and CSRF rejection passed. Portable PHP was used locally; a writable temporary session directory resolved a local file-write restriction without changing source code.

ReserveIT requester, staff and administrator workflows passed, including room/material requests, conflicting bookings, approval, cancellation, completion, inventory and penalties. The application also served its login page over HTTP. The source access suite checks database accounts independently. Actual runtime INSERT/UPDATE/DELETE affected a temporary material row, and a maintenance UPDATE affected a row; both were rolled back.

Successful data-changing SQL used temporary fixtures and rollback where supported. Failed statements were recognized as permission/connection denials, not arbitrary errors. SQLSTATE 42501 appears in available PostgreSQL privilege-error output. ReserveIT original access logs record expected/actual denial but do not preserve the full PostgreSQL exception text.

Some ReserveIT access checks use WHERE false. Those demonstrate statement-level privilege enforcement and do not prove that a row was changed. The matrix labels those cases; separate application state assertions and actual-row supplementary operations provide the data-changing evidence.

Verification table

Each row identifies the actual login, operation, expected/actual result and evidence. The complete SQL, target database and evidence reference are also in verification-results.csv. G02 sql-N.txt contains the exact submitted SQL/output; G07 JSON item N contains statement/output; G11 CASE N is paired with the original source and actual access log. I-prefixed rows are real connection-isolation attempts.

ID / login	SQL operation	Expected / actual	Evidence
G02-1 event_app	Read role	Allowed / Allowed	sql-1.txt
G02-2 event_app	Read user_account	Allowed / Allowed	sql-2.txt
G02-3 event_app	Read venue	Allowed / Allowed	sql-3.txt
G02-4 event_app	Read participant	Allowed / Allowed	sql-4.txt
G02-5 event_app	Read event	Allowed / Allowed	sql-5.txt
G02-6 event_app	Read event_schedule	Allowed / Allowed	sql-6.txt
G02-7 event_app	Read event_assignment	Allowed / Allowed	sql-7.txt
G02-8 event_app	Read attendance	Allowed / Allowed	sql-8.txt
G02-9 event_app	Read audit_log	Allowed / Allowed	sql-9.txt
G02-10 event_app	Insert into 8 tables using generated sequences	Allowed / Allowed	sql-10.txt
G02-11 event_app	Update venue	Allowed / Allowed	sql-11.txt

ID / login	SQL operation	Expected / actual	Evidence
G02-12 event_app	Update participant	Allowed / Allowed	sql-12.txt
G02-13 event_app	Update event	Allowed / Allowed	sql-13.txt
G02-14 event_app	Update event_schedule	Allowed / Allowed	sql-14.txt
G02-15 event_app	Update event_assignment	Allowed / Allowed	sql-15.txt
G02-16 event_app	Update attendance	Allowed / Allowed	sql-16.txt
G02-17 event_app	Update user_account	Allowed / Allowed	sql-17.txt
G02-18 event_app	Delete attendance	Allowed / Allowed	sql-18.txt
G02-19 event_app	Delete event_schedule	Allowed / Allowed	sql-19.txt
G02-20 event_app	Delete event_assignment	Allowed / Allowed	sql-20.txt
G02-21 event_app	Delete event	Allowed / Allowed	sql-21.txt
G02-22 event_app	Delete participant	Allowed / Allowed	sql-22.txt
G02-23 event_app	Delete venue	Allowed / Allowed	sql-23.txt
G02-24 event_app	CREATE TABLE school_event_management_db.activity4_probe(id int);	Denied / Denied	sql-24.txt
G02-25 event_app	ALTER TABLE school_event_management_db.venue ADD COLUMN activity4_probe int;	Denied / Denied	sql-25.txt
G02-26 event_app	DROP TABLE school_event_management_db.venue;	Denied / Denied	sql-26.txt
G02-27 event_app	TRUNCATE school_event_management_db.audit_log;	Denied / Denied	sql-27.txt
G02-28 event_app	UPDATE school_event_management_db.audit_log SET action_performed='Updated';	Denied / Denied	sql-28.txt
G02-29 event_app	DELETE FROM school_event_management_db.audit_log;	Denied / Denied	sql-29.txt
G02-30 event_app	INSERT INTO school_event_management_db.role(role_name) VALUES ('probe');	Denied / Denied	sql-30.txt
G02-31 event_app	DELETE FROM school_event_management_db.user_account;	Denied / Denied	sql-31.txt
G02-32 event_app	SELECT * FROM school_event_management_db.flyway_schema_history;	Denied / Denied	sql-32.txt
G02-33 event_app	CREATE DATABASE grp02_permission_probe;	Denied / Denied	sql-33.txt
G02-34 grp02_readonly	Readonly SELECT role	Allowed / Allowed	sql-34.txt
G02-35 grp02_readonly	Readonly SELECT venue	Allowed / Allowed	sql-35.txt

ID / login	SQL operation	Expected / actual	Evidence
G02-36 grp02_readonly	Readonly SELECT event	Allowed / Allowed	sql-36.txt
G02-37 grp02_readonly	INSERT INTO venue(venue_name,location) VALUES ('probe','probe');	Denied / Denied	sql-37.txt
G02-38 grp02_readonly	UPDATE venue SET location='probe';	Denied / Denied	sql-38.txt
G02-39 grp02_readonly	DELETE FROM venue;	Denied / Denied	sql-39.txt
G02-40 grp02_readonly	SELECT * FROM user_account;	Denied / Denied	sql-40.txt
G02-41 grp02_readonly	CREATE TABLE school_event_management_db.activity4_probe(id int);	Denied / Denied	sql-41.txt
G02-42 grp02_readonly	CREATE DATABASE grp02_permission_probe;	Denied / Denied	sql-42.txt
G02-43 event_owner	Migration owner create alter drop	Allowed / Allowed	sql-43.txt
G02-44 event_app	Unrelated database connect denied	Denied / Denied	sql-44.txt
G02-45 event_app	Unrelated database connect denied	Denied / Denied	sql-45.txt
G02-46 event_app	Unrelated database connect denied	Denied / Denied	sql-46.txt
G02-47 event_owner	Unrelated database connect denied	Denied / Denied	sql-47.txt
G02-48 event_owner	Unrelated database connect denied	Denied / Denied	sql-48.txt
G02-49 event_owner	Unrelated database connect denied	Denied / Denied	sql-49.txt
G02-50 grp02_readonly	Unrelated database connect denied	Denied / Denied	sql-50.txt
G02-51 grp02_readonly	Unrelated database connect denied	Denied / Denied	sql-51.txt
G02-52 grp02_readonly	Unrelated database connect denied	Denied / Denied	sql-52.txt
G07-1 grp07_app	SELECT count(*) FROM public.department;	Allowed / Allowed	SQL JSON #1
G07-2 grp07_readonly	SELECT count(*) FROM public.department;	Allowed / Allowed	SQL JSON #2
G07-3 grp07_app	SELECT count(*) FROM public.position;	Allowed / Allowed	SQL JSON #3
G07-4 grp07_readonly	SELECT count(*) FROM public.position;	Allowed / Allowed	SQL JSON #4
G07-5 grp07_app	SELECT count(*) FROM public.applicant;	Allowed / Allowed	SQL JSON #5
G07-6 grp07_readonly	SELECT count(*) FROM public.applicant;	Allowed / Allowed	SQL JSON #6
G07-7 grp07_app	SELECT count(*) FROM public.application_status;	Allowed / Allowed	SQL JSON #7

ID / login	SQL operation	Expected / actual	Evidence
G07-8 grp07_readonly	SELECT count(*) FROM public.application_status;	Allowed / Allowed	SQL JSON #8
G07-9 grp07_app	SELECT count(*) FROM public.application;	Allowed / Allowed	SQL JSON #9
G07-10 grp07_readonly	SELECT count(*) FROM public.application;	Allowed / Allowed	SQL JSON #10
G07-11 grp07_app	SELECT count(*) FROM public.interview;	Allowed / Allowed	SQL JSON #11
G07-12 grp07_readonly	SELECT count(*) FROM public.interview;	Allowed / Allowed	SQL JSON #12
G07-13 grp07_app	SELECT count(*) FROM public.requirement;	Allowed / Allowed	SQL JSON #13
G07-14 grp07_readonly	SELECT count(*) FROM public.requirement;	Allowed / Allowed	SQL JSON #14
G07-15 grp07_app	SELECT count(*) FROM public.applicant_requirement;	Allowed / Allowed	SQL JSON #15
G07-16 grp07_readonly	SELECT count(*) FROM public.applicant_requirement;	Allowed / Allowed	SQL JSON #16
G07-17 grp07_app	Insert valid fixture into operational tables	Allowed / Allowed	SQL JSON #17
G07-18 grp07_app	Insert valid fixture into operational tables and update fixture rows	Allowed / Allowed	SQL JSON #18
G07-19 grp07_app	Insert valid fixture into operational tables and delete fixture rows	Allowed / Allowed	SQL JSON #19
G07-20 grp07_app	CREATE TABLE public.activity4_probe(id int);	Denied / Denied	SQL JSON #20
G07-21 grp07_app	ALTER TABLE public.department ADD COLUMN activity4_probe int;	Denied / Denied	SQL JSON #21
G07-22 grp07_app	DROP TABLE public.applicant;	Denied / Denied	SQL JSON #22
G07-23 grp07_app	CREATE DATABASE grp07_permission_probe;	Denied / Denied	SQL JSON #23
G07-24 grp07_app	CREATE ROLE grp07_permission_probe;	Denied / Denied	SQL JSON #24
G07-25 grp07_app	INSERT INTO public.application_status(status_name) VALUES('Probe');	Denied / Denied	SQL JSON #25
G07-26 grp07_readonly	INSERT INTO public.department(department_name) VALUES('Readonly probe');	Denied / Denied	SQL JSON #26
G07-27 grp07_readonly	UPDATE public.department SET description='probe';	Denied / Denied	SQL JSON #27
G07-28 grp07_readonly	DELETE FROM public.department;	Denied / Denied	SQL JSON #28
G07-29 grp07_readonly	CREATE TABLE public.activity4_probe(id int);	Denied / Denied	SQL JSON #29
G07-30 grp07_readonly	CREATE DATABASE grp07_permission_probe;	Denied / Denied	SQL JSON #30
G07-31 grp07_readonly	CREATE ROLE grp07_permission_probe;	Denied / Denied	SQL JSON #31

ID / login	SQL operation	Expected / actual	Evidence
G07-32 grp07_owner	CREATE TABLE public.activity4_owner_probe(id int); ALTER TABLE public.activity4_owner_probe ADD COLUMN note text; DROP TABLE public.activity4_owner_probe;	Allowed / Allowed	SQL JSON #32
G07-33 grp07_app	Connect to dropboxed_db	Denied / Denied	SQL JSON #33
G07-34 grp07_app	Connect to group3_Pawpaw	Denied / Denied	SQL JSON #34
G07-35 grp07_app	Connect to grp99_gymdb	Denied / Denied	SQL JSON #35
G07-36 grp07_app	Connect to grp02_event_management_db	Denied / Denied	SQL JSON #36
G07-37 grp07_readonly	Connect to dropboxed_db	Denied / Denied	SQL JSON #37
G07-38 grp07_readonly	Connect to group3_Pawpaw	Denied / Denied	SQL JSON #38
G07-39 grp07_readonly	Connect to grp99_gymdb	Denied / Denied	SQL JSON #39
G07-40 grp07_readonly	Connect to grp02_event_management_db	Denied / Denied	SQL JSON #40
G07-41 grp07_owner	Connect to dropboxed_db	Denied / Denied	SQL JSON #41
G07-42 grp07_owner	Connect to group3_Pawpaw	Denied / Denied	SQL JSON #42
G07-43 grp07_owner	Connect to grp99_gymdb	Denied / Denied	SQL JSON #43
G07-44 grp07_owner	Connect to grp02_event_management_db	Denied / Denied	SQL JSON #44
G11-1 reserveit_app	read rooms	Allowed / Allowed	access log CASE 1
G11-2 reserveit_app	read login data (password_hash) for sign-in	Allowed / Allowed	access log CASE 2
G11-3 reserveit_app	register a new user	Allowed / Allowed	access log CASE 3
G11-4 reserveit_app	suspend or reactivate an account (status column) [WHERE false]	Allowed / Allowed	access log CASE 4
G11-5 reserveit_app	approve a reservation (status, staff_id) [WHERE false]	Allowed / Allowed	access log CASE 5
G11-6 reserveit_app	mark a penalty paid [WHERE false]	Allowed / Allowed	access log CASE 6
G11-7 reserveit_app	edit and delete rooms [WHERE false]	Allowed / Allowed	access log CASE 7
G11-8 reserveit_app	delete a material [WHERE false]	Allowed / Allowed	access log CASE 8
G11-9 reserveit_app	change a user's password hash [WHERE false]	Denied / Denied	access log CASE 9
G11-10 reserveit_app	change a reservation's purpose or date [WHERE false]	Denied / Denied	access log CASE 10

ID / login	SQL operation	Expected / actual	Evidence
G11-11 reserveit_app	change a penalty amount [WHERE false]	Denied / Denied	access log CASE 11
G11-12 reserveit_app	delete reservations [WHERE false]	Denied / Denied	access log CASE 12
G11-13 reserveit_app	delete penalties [WHERE false]	Denied / Denied	access log CASE 13
G11-14 reserveit_app	delete user accounts [WHERE false]	Denied / Denied	access log CASE 14
G11-15 reserveit_app	delete staff accounts [WHERE false]	Denied / Denied	access log CASE 15
G11-16 reserveit_app	delete departments [WHERE false]	Denied / Denied	access log CASE 16
G11-17 reserveit_app	edit reservation line items [WHERE false]	Denied / Denied	access log CASE 17
G11-18 reserveit_app	empty a table	Denied / Denied	access log CASE 18
G11-19 reserveit_app	create a table	Denied / Denied	access log CASE 19
G11-20 reserveit_app	drop a table	Denied / Denied	access log CASE 20
G11-21 reserveit_app	create a role	Denied / Denied	access log CASE 21
G11-22 reserveit_app	create a database	Denied / Denied	access log CASE 22
G11-23 reserveit_reporter	read reservations	Allowed / Allowed	access log CASE 23
G11-24 reserveit_reporter	read penalties	Allowed / Allowed	access log CASE 24
G11-25 reserveit_reporter	read user names and status	Allowed / Allowed	access log CASE 25
G11-26 reserveit_reporter	read password hashes	Denied / Denied	access log CASE 26
G11-27 reserveit_reporter	read staff password hashes	Denied / Denied	access log CASE 27
G11-28 reserveit_reporter	read user email addresses	Denied / Denied	access log CASE 28
G11-29 reserveit_reporter	insert data	Denied / Denied	access log CASE 29
G11-30 reserveit_reporter	update data [WHERE false]	Denied / Denied	access log CASE 30
G11-31 reserveit_reporter	delete data [WHERE false]	Denied / Denied	access log CASE 31
G11-32 reserveit_reporter	create a table	Denied / Denied	access log CASE 32
G11-33 reserveit_dba	read everything	Allowed / Allowed	access log CASE 33
G11-34 reserveit_dba	correct data [WHERE false]	Allowed / Allowed	access log CASE 34

ID / login	SQL operation	Expected / actual	Evidence
G11-35 reserveit_dba	remove bad records [WHERE false]	Allowed / Allowed	access log CASE 35
G11-36 reserveit_dba	change the table structure	Denied / Denied	access log CASE 36
G11-37 reserveit_dba	create a table	Denied / Denied	access log CASE 37
G11-38 reserveit_dba	drop a table	Denied / Denied	access log CASE 38
G11-39 reserveit_dba	create a role	Denied / Denied	access log CASE 39
G11-40 reserveit_owner	create and change tables	Allowed / Allowed	access log CASE 40
G11-41 reserveit_owner	create a role	Denied / Denied	access log CASE 41
G11-42 reserveit_owner	create a database	Denied / Denied	access log CASE 42
G11-I1 reserveit_app	Connect to dropboxed_db	Denied / Denied	isolation JSON #1
G11-I2 reserveit_app	Connect to group3_Pawpaw	Denied / Denied	isolation JSON #2
G11-I3 reserveit_app	Connect to grp99_gymdb	Denied / Denied	isolation JSON #3
G11-I4 reserveit_app	Connect to grp02_event_management_db	Denied / Denied	isolation JSON #4
G11-I5 reserveit_app	Connect to grp07_applicantdb	Denied / Denied	isolation JSON #5
G11-I6 reserveit_owner	Connect to dropboxed_db	Denied / Denied	isolation JSON #6
G11-I7 reserveit_owner	Connect to group3_Pawpaw	Denied / Denied	isolation JSON #7
G11-I8 reserveit_owner	Connect to grp99_gymdb	Denied / Denied	isolation JSON #8
G11-I9 reserveit_owner	Connect to grp02_event_management_db	Denied / Denied	isolation JSON #9
G11-I10 reserveit_owner	Connect to grp07_applicantdb	Denied / Denied	isolation JSON #10
G11-I11 reserveit_reporter	Connect to dropboxed_db	Denied / Denied	isolation JSON #11
G11-I12 reserveit_reporter	Connect to group3_Pawpaw	Denied / Denied	isolation JSON #12
G11-I13 reserveit_reporter	Connect to grp99_gymdb	Denied / Denied	isolation JSON #13
G11-I14 reserveit_reporter	Connect to grp02_event_management_db	Denied / Denied	isolation JSON #14
G11-I15 reserveit_reporter	Connect to grp07_applicantdb	Denied / Denied	isolation JSON #15
G11-I16 reserveit_dba	Connect to dropboxed_db	Denied / Denied	isolation JSON #16

ID / login	SQL operation	Expected / actual	Evidence
G11-I17 reserveit_dba	Connect to group3_Pawpaw	Denied / Denied	isolation JSON #17
G11-I18 reserveit_dba	Connect to grp99_gymdb	Denied / Denied	isolation JSON #18
G11-I19 reserveit_dba	Connect to grp02_event_management_db	Denied / Denied	isolation JSON #19
G11-I20 reserveit_dba	Connect to grp07_applicantdb	Denied / Denied	isolation JSON #20

Additional evidence distinctions

Account-attribute, table-ownership and PUBLIC CONNECT assertions in the original ReserveIT access log are configuration assertions, separate from the SQL operation rows above. Group 2 before-isolation-fix evidence retains the original successful cross-database logins; the final matrix records denial after the correction. No initial failure was discarded to imply the first setup was already correct.

Issues and attempted fixes

Project	Finding and action	Remaining limitation
G02	Role documentation names public and proposed role names that differ from actual schema/accounts. Implemented separate NOLOGIN roles and runtime/read-only memberships locally. Documented constraint-test file was absent.	Owner documentation/script correction is pending. Manual browser-screen and accessibility checks were not performed.
G07	Role setup was left for Activity 4. Added runtime/read-only groups and logins; restricted the status lookup to SELECT; added account-specific connection isolation. Local PHP session writes required Windows Temp.	Basic HTTP CRUD was tested; no claim that every UI interaction or all application role scenarios were covered.
G10	Inspected repository and ZIP. Both active application versions use MySQLi while PostgreSQL/PDO remains planned. No source rewrite was attempted.	PostgreSQL application functionality and account permissions are not verified. Owning group must complete migration.
G11	Supplied roles/grants worked. Added local account-only cross-database connection restrictions. Kept WHERE false checks distinct from actual data changes.	Smoke tests intentionally retain development records. No manual browser accessibility walkthrough was performed.

Earlier tracker records and whole-class coverage

The class sheet also records earlier G09 results for G01 Dropboxed and G03 Pawpaw Clinic. Their existing objects and broad direct runtime CRUD grants were inspected administratively. Original per-operation logs and account passwords were not recovered. New login attempts without local passwords could not authenticate. No credentials were reset and no successful restricted-account retest is claimed.

Those accounts are non-superusers, but NOLOGIN memberships, separate reporting verification and a completed least-privilege review are not established by the recovered evidence. PUBLIC CONNECT remains enabled on those earlier databases. Dropboxed local configuration was observed to name postgres, so its ordinary application connection needs review before claiming it uses the restricted login. Supplementary folders disclose these gaps; existing tracker success labels were not silently replaced.

This package is not proof that every other class project has been tested. The remaining class inventory and the earlier evidence gaps still need attention before checking the professor's entire submission checklist as complete.

Feedback and tracker preparation

Feedback drafts are provided in feedback-ready-to-send.md. They have not been sent, so the feedback status is No and no sent date is invented. Send each message to the owning group and enter the actual delivery date afterwards. No tool was used to message other people.

The existing class sheet has a six-column activity log. The official assignment requires an Activity 4 Log with Project, Repository cloned, Database setup result, Access-control folder link, Access-control status, Feedback sent to owners, PDF progress document and Last updated. Activity4-Log-paste-ready.tsv supplies those columns for the four current projects. Legacy short notes are provided separately. No sheet edit was performed.

Use Privileges tested for the working PostgreSQL projects. SHIORI remains Not started for PostgreSQL access control and Failed for setup. Do not mark Completed merely because the files have been assembled.

AI usage declaration

We used ChatGPT/Codex to help understand the instructions, explain PostgreSQL roles and permissions, prepare SQL and test scripts, troubleshoot setup, and organize the report. The assistant also ran the documented commands in our local test environment. Results were checked against actual database output, application responses and test logs. Missing evidence and incomplete work are stated.

Conclusion and submission actions

The working PostgreSQL projects have documented access-control plans and evidence for allowed and denied operations. SHIORI has a documented setup blocker. The report and repository package preserve those differences. The final submission still requires sending feedback, pasting the prepared tracker rows, confirming the repository/PDF links are published, and addressing the remaining class coverage and older-project evidence gaps.

Submit the PDF and the Batumbakal Gym repository link to the professor. Keep the SQL scripts, plans and evidence under db-access-control/ in that repository. Editable report text is provided as Markdown; no LibreOffice installation or DOCX conversion was used.

References

Official supplied COMP018 Activity 4 instructions; project repositories and local snapshots listed above; actual terminal/API/SQL logs in the evidence folders. PostgreSQL connection-policy reference: https://www.postgresql.org/docs/18/auth-pg-hba-conf.html